

Code Analysis & Complexity

Non-Recursive — Brute Force Algorithm

Code Structure

1 is Triangle(int a, int b, int c)

Validates the triangle inequality theorem: the sum of any two sides must be strictly greater than the third side.

```
if (a + b > c && b + c > a && c + a > b) return 1;
```

All three conditions must hold simultaneously for a valid triangle.

2 Brute force (int A[], int n)

Returns 1 as soon as a valid triangle is found, or 0 if none exists.

3 main()

Tests two hard-coded arrays and prints the results:

- {10, 50, 1} → Expected: 0 (no valid triangle)
- {10, 2, 5, 1, 8, 20} → Expected: 1 (valid triangle exists)

Complexity Analysis:

1)Time Complexity

The algorithm uses three nested loops over indices i, j, and k, producing the following work:

Component	Expression	Big-O
Outer loop (i)	$0 \rightarrow n - 3$	$O(n)$
Middle loop (j)	$i+1 \rightarrow n - 2$	$O(n)$
Inner loop (k)	$j+1 \rightarrow n - 1$	$O(n)$
Is Triangle call	Constant — 3 comparisons	$O(1)$
Overall (worst case)	$C(n, 3) = n(n-1)(n-2) / 6$	$O(n^3)$

In the best case (a valid triangle is found early), the algorithm exits immediately — $O(1)$. In the worst case (no triangle or it is found at the very last triplet), all $C(n, 3)$ combinations are visited — $O(n^3)$.

2) Space Complexity

The algorithm operates entirely in-place, using only loop-index variables. & since the number of Variables is constant, the space complexity is constant $O(1)$

Memory Usage	Details	Complexity
Loop indices (i, j, k)	Three integer variables	$O(1)$
Function call stack	isTriangle is non-recursive	$O(1)$
Input array	Read-only, passed by pointer	$O(n)$ — caller-owned
Total space	—	$O(1)$

Optimized Alternative — $O(n \log n)$

Approach	Time	Space	Notes
Brute force (current)	$O(n^3)$	$O(1)$	Works on unsorted arrays
Sort + single pass	$O(n \log n)$	$O(1)$	Requires sorting; far faster